

Full Stack Web Development

Solved Question Paper — Mid Semester Examination 2023
MCA | Semester III | Paper Code: TMC-304 | Max. Marks: 50

Note: Each question below is answered in full (both the (a) and (b) options), even though the exam only requires one, so this document can double as a complete revision set for the syllabus. Marks-appropriate depth is indicated, with extra explanatory content added for clarity and retention.

Q1 (a)

10 Marks | CO2

What is the use of geolocation feature of HTML5? Write a program to display user's longitude and latitude coordinates on the web page.

Answer:

The **Geolocation API** is a feature introduced in HTML5 that allows a web application, with the user's explicit permission, to access the geographical location of the device on which the browser is running. It is exposed through the `navigator.geolocation` object in JavaScript and is one of the key APIs that make modern location-aware web apps possible.

Uses / Applications of Geolocation API

- Showing a user's current position on a map (e.g. Google Maps style apps).
- Finding nearby places — restaurants, ATMs, hospitals, stores — based on current location.
- Location-based services such as ride-hailing apps, weather apps, and delivery tracking.
- Auto-filling address fields during checkout/registration.
- Geo-tagging photos, posts, or check-ins on social media.
- Navigation and turn-by-turn direction systems.
- Local content personalization — currency, language, or offers based on region.

How it Works

The browser determines the position using a combination of sources such as GPS (most accurate, used on mobile devices), Wi-Fi access point triangulation, cell tower triangulation, and IP address lookup (least accurate, used as a fallback on desktops). Because location is sensitive personal data, the API is available only on secure origins (HTTPS) and the browser always shows a permission prompt before sharing it with the page.

Key Methods of the Geolocation Object

Method	Description
<code>getCurrentPosition(success, error, options)</code>	Retrieves the device's current position a single time and invokes the success callback with a <code>Position</code> object, or the error callback on failure.
<code>watchPosition(success, error, options)</code>	Registers a handler that is called automatically every time the device's position changes — useful for continuous tracking such as live navigation.
<code>clearWatch(id)</code>	Stops the continuous watch that was started by <code>watchPosition()</code> , using the watch ID it returned.

Important Properties of the Position Object

- `coords.latitude` — latitude in decimal degrees.
- `coords.longitude` — longitude in decimal degrees.
- `coords.accuracy` — accuracy of latitude/longitude in metres.
- `coords.altitude`, `coords.speed`, `coords.heading` — optional, if available.

- timestamp — time the position was captured.

Program: Displaying Latitude & Longitude

```
<!DOCTYPE html>
<html>
<head>
  <title>HTML5 Geolocation Example</title>
</head>
<body>
  <h2>Your Current Location</h2>
  <button onclick="getLocation()">Get Location</button>
  <p id="output"></p>

  <script>
    var output = document.getElementById("output");

    function getLocation() {
      if (navigator.geolocation) {
        navigator.geolocation.getCurrentPosition(showPosition, showError);
      } else {
        output.innerHTML = "Geolocation is not supported by this browser.";
      }
    }

    function showPosition(position) {
      var lat = position.coords.latitude;
      var lon = position.coords.longitude;
      output.innerHTML =
        "Latitude: " + lat + "<br>" +
        "Longitude: " + lon + "<br>" +
        "Accuracy: " + position.coords.accuracy + " metres";
    }

    function showError(error) {
      switch(error.code) {
        case error.PERMISSION_DENIED:
          output.innerHTML = "User denied the request for Geolocation.";
          break;
        case error.POSITION_UNAVAILABLE:
          output.innerHTML = "Location information is unavailable.";
          break;
        case error.TIMEOUT:
          output.innerHTML = "The request to get user location timed out.";
          break;
        default:
          output.innerHTML = "An unknown error occurred.";
      }
    }
  </script>
</body>
</html>
```

Note: When the page is opened and the button clicked, the browser first shows a permission dialog. Once allowed, the coordinates are fetched asynchronously and written into the page via the success callback.

What is MathML in HTML5? Print the following equations using HTML5 code: (i) $a^2-b^2/c^2-d^2=e/f$ (ii) $d/dx x^3 + \int x^2 dx$

Answer:

MathML (Mathematical Markup Language) is an XML-based markup language designed to describe mathematical notation and structure so that it can be displayed correctly inside a web page. HTML5 natively supports embedding MathML directly inside the HTML document (inside a `<math>` element), so complex mathematical expressions — fractions, powers, roots, integrals, matrices — can be rendered by the browser itself, without images or external plugins.

Why MathML is Needed

- Plain HTML has no reliable way to represent fractions, superscripts stacked with subscripts, roots, or integral signs in a semantically correct, accessible way.
- MathML expressions are searchable, selectable text — not images — so they scale and print cleanly and can be read by screen readers.
- It is a W3C standard and integrates with HTML5's DOM, so it can be styled with CSS and manipulated with JavaScript like any other element.

Common MathML Elements

Tag	Meaning
<code><math></code>	Root element that wraps a MathML expression.
<code><mrow></code>	Groups a horizontal row of sub-expressions.
<code><mi></code>	Identifier — a variable name, e.g. x, y, a.
<code><mn></code>	A number, e.g. 2, 3.
<code><mo></code>	An operator, e.g. +, −, =, ∫.
<code><msup></code>	Superscript — base and exponent, used for powers.
<code><msub></code>	Subscript.
<code><mfrac></code>	A fraction — numerator and denominator.
<code><msqrt></code> / <code><mroot></code>	Square root / nth root.

(i) Program for $a^2-b^2/c^2-d^2 = e/f$

```
<!DOCTYPE html>
<html>
<body>
  <h3>Equation 1</h3>
  <math xmlns="http://www.w3.org/1998/Math/MathML">
    <mrow>
      <mfrac>
        <mrow>
          <msup><mi>a</mi><mn>2</mn></msup>
          <mo>-</mo>
          <msup><mi>b</mi><mn>2</mn></msup>
        </mrow>
        <mrow>
          <msup><mi>c</mi><mn>2</mn></msup>
          <mo>-</mo>
          <msup><mi>d</mi><mn>2</mn></msup>
        </mrow>
      </mfrac>
    </mrow>
  </math>
```

```

        <mo>=</mo>
        <mfrac>
            <mi>e</mi>
            <mi>f</mi>
        </mfrac>
    </mrow>
</math>
</body>
</html>

```

(ii) Program for $\frac{d}{dx}(x^3) + \int x^2 dx$

```

<!DOCTYPE html>
<html>
<body>
    <h3>Equation 2</h3>
    <math xmlns="http://www.w3.org/1998/Math/MathML">
        <mrow>
            <mfrac>
                <mi>d</mi>
                <mrow><mi>d</mi><mi>x</mi></mrow>
            </mfrac>
            <msup><mi>x</mi><mn>3</mn></msup>
            <mo>+</mo>
            <mo>&#8747;</mo><mo>&#8747;</mo>
            <msup><mi>x</mi><mn>2</mn></msup>
            <mi>d</mi><mi>x</mi>
        </mrow>
    </math>
</body>
</html>

```

Both snippets can be placed inside the `<body>` of any HTML5 page; MathML-capable browsers render the `<math>` block as properly typeset mathematics (stacked fraction bars, raised exponents, and integral signs) rather than as plain inline text.

Q2 (a)**10 Marks | CO2**

What do you mean by transform in CSS? Explain all the methods of 2d and 3d transform in detail. 3d Transform an element with perspective view into 45 degrees and scale double from the original size, where the origin of the transformation is center-center.

Answer:

The CSS **transform** property lets you visually manipulate an element in 2D or 3D space — moving, rotating, scaling, or skewing it — without affecting the normal flow of the surrounding layout. It is widely used for animations, hover effects, card flips, and interactive UI components because transformed elements can be rendered efficiently by the GPU.

2D Transform Functions

Function	Effect
translate(x, y)	Moves the element along the X and Y axis by the given distance.
rotate(angle)	Rotates the element clockwise by the given angle (deg).
scale(sx, sy)	Resizes the element by a factor along X and Y axis.
skew(x-angle, y-angle)	Slants/tilts the element along the X and/or Y axis.
matrix(a,b,c,d,e,f)	Combines translate, rotate, scale and skew into one 2D transformation using a 6-value matrix.

3D Transform Functions

Function	Effect
translate3d(x, y, z)	Moves the element in 3D space along X, Y and Z axis.
translateZ(z)	Moves the element only along the Z axis (toward/away from viewer).
rotateX(angle) / rotateY(angle) / rotateZ(angle)	Rotates the element around the X, Y or Z axis respectively.
rotate3d(x,y,z,angle)	Rotates around a custom 3D vector by the given angle.
scale3d(sx,sy,sz)	Scales the element along all three axes.
scaleZ(sz)	Scales the element along the Z axis only.
perspective(n)	Defines the distance from the viewer to the z=0 plane, giving 3D transforms a sense of depth — smaller n = stronger effect.
matrix3d(...16 values...)	General 4x4 matrix combining every 3D transform into one.

Supporting Properties

- **transform-origin** — sets the point about which the transform is applied (default: center center / 50% 50%).
- **transform-style: preserve-3d** — makes child elements keep their own 3D positioning instead of being flattened.
- **perspective** (as a property on the parent) — an alternative way to set perspective for all 3D children at once.
- **backface-visibility** — controls whether the back of a rotated element is visible.

Required Program: Perspective 45°, Scale Double, Origin Center-Center

The element is given a perspective view, rotated to a 45-degree angle in 3D and scaled to double its original size, with the transform origin fixed at the exact center of the element:

```
<!DOCTYPE html>
<html>
<head>
<style>
  .box {
    width: 150px;
    height: 150px;
    background: linear-gradient(135deg, #4facfe, #00f2fe);
    margin: 100px;
    transform-origin: center center; /* pivot at the exact center */
    transform: perspective(500px) rotateY(45deg) scale(2);
    transition: transform 0.6s ease;
  }
</style>
</head>
<body>
  <div class="box"></div>
</body>
</html>
```

Here `perspective(500px)` creates the 3D viewing depth, `rotateY(45deg)` tilts the box 45° in 3D space, and `scale(2)` doubles its size — all pivoting exactly around the box's center because of `transform-origin: center center`.

What do you mean by Gradient in CSS? Explain the types of gradients and apply the gradient methods on the <div> element?

Answer:

A **CSS gradient** is a smooth transition between two or more colors, generated by the browser itself (not an image file). Gradients are set as a value of the background / background-image property, which keeps pages lightweight and lets the gradient scale perfectly to any element size.

Types of CSS Gradients

Type	Description
linear-gradient()	Colors transition along a straight line, in a direction you specify (angle or keyword like 'to right').
radial-gradient()	Colors radiate outward from a center point in a circular or elliptical shape.
conic-gradient()	Colors rotate around a center point like a color wheel / pie chart, instead of radiating outward.
repeating-linear-gradient()	A linear gradient pattern that repeats to fill the element (creates stripes).
repeating-radial-gradient()	A radial gradient pattern that repeats outward (creates concentric rings).

Applying Gradients on a <div> Element

```
<!DOCTYPE html>
<html>
<head>
<style>
  div {
    width: 260px;
    height: 120px;
    margin: 15px;
    border-radius: 8px;
    color: #fff;
    font-family: sans-serif;
    display: flex;
    align-items: center;
    justify-content: center;
  }

  /* 1. Linear gradient - 45 degree direction */
  .linear {
    background: linear-gradient(45deg, #ff512f, #f09819);
  }

  /* 2. Radial gradient - circular, from center */
  .radial {
    background: radial-gradient(circle, #43cea2, #185a9d);
  }

  /* 3. Conic gradient - color wheel style */
  .conic {
    background: conic-gradient(from 0deg, red, yellow, lime, aqua, blue, magenta, red);
  }

  /* 4. Repeating linear gradient - stripes */
  .repeating {
    background: repeating-linear-gradient(45deg, #333 0 10px, #eee 10px 20px);
    color: #000;
  }
}
```

```
</style>
</head>
<body>
  <div class="linear">Linear Gradient</div>
  <div class="radial">Radial Gradient</div>
  <div class="conic">Conic Gradient</div>
  <div class="repeating">Repeating Gradient</div>
</body>
</html>
```

Multiple color stops can also be given specific positions, e.g. `linear-gradient(to right, red 0%, yellow 50%, green 100%)`, to control exactly where each color begins and ends.

What do you understand by full stack development? Explain the scope and benefit of full stack development in detail.

Answer:

Full stack development refers to the practice of building both the front-end (client-side, what the user sees and interacts with) and the back-end (server-side, business logic, database, APIs) of a web application. A **full stack developer** is someone comfortable working across the entire 'stack' of technologies needed to take a web application from concept to a fully working, deployed product — UI design, server logic, database design, APIs, and often basic DevOps/deployment as well.

Scope of Full Stack Development

- **Front-end scope:** HTML, CSS, JavaScript and front-end frameworks/libraries such as React, Angular or Vue — covering layout, styling, responsiveness and client-side interactivity.
- **Back-end scope:** server-side languages/frameworks such as Node.js/Express, Django, Spring Boot or PHP — covering business logic, authentication, and REST/GraphQL APIs.
- **Database scope:** designing and querying relational databases (MySQL, PostgreSQL) or NoSQL databases (MongoDB) to persist application data.
- **Version control & collaboration:** using Git/GitHub for source control and team collaboration.
- **Deployment & DevOps basics:** hosting on cloud platforms (AWS, Azure, Heroku, Vercel), using CI/CD pipelines, containers (Docker), and basic server administration.
- **Growing scope:** with the rise of cloud-native and mobile-first apps, full stack developers are increasingly expected to know cloud services, mobile responsiveness/PWAs, and basic security practices, which greatly widens their employability across startups and enterprises alike.

Benefits of Full Stack Development

- **Faster development:** one person/team can handle an entire feature — from UI to database — without waiting on hand-offs between separate front-end and back-end teams.
- **Better problem solving:** understanding the whole system makes it easier to debug issues that span layers (e.g. a slow API call affecting the UI).
- **Cost effective:** hiring fewer, versatile developers reduces team size and communication overhead, which is especially valuable for startups.
- **Flexibility:** full stack developers can move between tasks as project priorities shift, and can prototype an entire application single-handedly.
- **Improved collaboration:** since they understand both sides, they communicate more effectively with specialized front-end and back-end teams.
- **Greater career opportunities:** full stack skills are in high demand and open doors to a wide range of roles, from startups to large product companies.

Explain the components of full stack development in detail with proper example.

Answer:

A full stack application is generally divided into three broad layers — the front end, the back end, and the database — supported by additional tooling for version control and deployment. Each component plays a specific role, as explained below with common tools/technologies used at each layer.

Component	Role & Example Technologies
Front-End (Client Side)	Builds everything the user directly sees and interacts with: layout, styling and interactivity. Example: HTML5 for structure, CSS3 for styling, JavaScript for behaviour, and frameworks like React.js or Angular for building dynamic, component-based interfaces.
Back-End (Server Side)	Handles the application's business logic, processes requests from the front-end, performs authentication/authorization, and talks to the database. Example: Node.js with Express.js, Python with Django/Flask, or Java with Spring Boot, exposing REST or GraphQL APIs.
Database	Stores and retrieves persistent application data such as users, products or orders. Example: relational databases like MySQL/PostgreSQL for structured data, or NoSQL databases like MongoDB for flexible, document-based data.
Server / Hosting Environment	The environment where the back-end application actually runs and serves requests. Example: Apache/Nginx web servers, or cloud platforms such as AWS EC2, Heroku, or Vercel.
APIs / Middleware	The contract through which the front-end and back-end communicate, plus middleware for tasks like logging, authentication checks and error handling. Example: RESTful JSON APIs, JWT-based auth middleware.
Version Control	Tracks code changes and enables team collaboration. Example: Git, hosted on GitHub/GitLab/Bitbucket.
Deployment / DevOps	Automates testing, building and releasing the application. Example: Docker for containerization, Jenkins/GitHub Actions for CI/CD pipelines.

Example: A Simple E-Commerce Application

- **Front-end:** a React.js storefront displays a product catalogue and shopping cart.
- **Back-end:** a Node.js/Express API handles 'add to cart', 'place order' and user login requests.
- **Database:** MongoDB stores product details, user accounts, and order records.
- **Deployment:** the front-end is hosted on Vercel, the back-end API on an AWS EC2 instance behind Nginx, with Git/GitHub used throughout for version control.

Together these components form the complete 'stack' — a request travels from the browser (front-end) to the API (back-end), which queries the database and sends the result back to be rendered on screen.

What is HTTP protocol? Explain the http request response header with proper diagram.

Answer:

HTTP (HyperText Transfer Protocol) is an application-layer protocol that defines the rules for how clients (web browsers) and servers communicate on the web. It is a **stateless, request-response** based protocol built on top of TCP/IP: the client sends an HTTP request asking for a resource (an HTML page, image, JSON data, etc.), and the server replies with an HTTP response containing the requested content or an error status.

Client-Server Request/Response Cycle

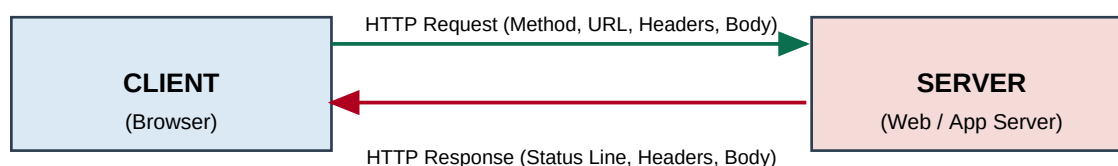


Fig. 1 — Basic HTTP request/response cycle between client and server

HTTP Request — Structure

An HTTP request consists of three parts:

- **Request line:** the HTTP method, the requested URL/path, and the HTTP version — e.g. `GET /index.html HTTP/1.1`.
- **Request headers:** metadata about the request (explained in the table below).
- **Request body** (optional): data sent to the server, typically used with POST/PUT requests (e.g. form data or JSON).

Common HTTP Request Headers

Header	Purpose
Host	The domain name of the server being requested (mandatory in HTTP/1.1).
User-Agent	Identifies the client software (browser/OS) making the request.
Accept	Tells the server which content types (e.g. text/html, application/json) the client can process.
Content-Type	The MIME type of the data being sent in the request body (used with POST/PUT).
Authorization	Carries credentials (e.g. a Bearer token) for authenticated requests.
Cookie	Sends previously stored cookies back to the server.

HTTP Response — Structure

An HTTP response likewise consists of three parts:

- **Status line:** HTTP version, a numeric status code, and a short reason phrase — e.g. `HTTP/1.1 200 OK`.
- **Response headers:** metadata describing the returned content (see table below).
- **Response body:** the actual content requested — HTML, JSON, an image, etc.

Common HTTP Response Headers

Header	Purpose
Content-Type	The MIME type of the returned body (e.g. text/html; charset=UTF-8).
Content-Length	The size of the response body, in bytes.
Set-Cookie	Instructs the browser to store a cookie for future requests.
Cache-Control	Directives about how the response may be cached.
Location	Used with redirect status codes to tell the client the new URL.
Server	Identifies the server software handling the request.

Common HTTP Status Code Classes

Range	Meaning
1xx	Informational — request received, continuing process.
2xx	Success — e.g. 200 OK, 201 Created.
3xx	Redirection — e.g. 301 Moved Permanently, 302 Found.
4xx	Client Error — e.g. 400 Bad Request, 404 Not Found, 401 Unauthorized.
5xx	Server Error — e.g. 500 Internal Server Error, 503 Service Unavailable.

Write a program to demonstrate the use of XMLHttpRequest (XHR) object for request response from server.

Answer:

The **XMLHttpRequest (XHR)** object is a built-in browser API that allows JavaScript to send HTTP requests to a server and receive a response **asynchronously**, without reloading the entire web page. It was the original technology behind AJAX (Asynchronous JavaScript and XML) and is the predecessor to the modern `fetch()` API.

Key Steps to Use XHR

- Create a new XHR object: `var xhr = new XMLHttpRequest();`
- Configure it with `xhr.open(method, url, async)`.
- Attach an `onreadystatechange` (or `onload`) handler to process the response.
- Send the request with `xhr.send()` (or `xhr.send(data)` for POST).
- Check `xhr.readyState === 4` and `xhr.status === 200` before reading `xhr.responseText`.

readyState Values

Value	State
0	UNSENT — <code>open()</code> has not been called yet.
1	OPENED — <code>open()</code> has been called.
2	HEADERS_RECEIVED — <code>send()</code> called, headers/status available.
3	LOADING — downloading response body.
4	DONE — the operation is complete.

Program: GET Request with XHR

```
<!DOCTYPE html>
<html>
<head><title>XMLHttpRequest Example</title></head>
<body>
  <h2>XHR Demo – Fetch data from server</h2>
  <button onclick="loadData()">Load Data</button>
  <div id="result"></div>

  <script>
    function loadData() {
      var xhr = new XMLHttpRequest();

      // Configure: GET request, asynchronous = true
      xhr.open("GET", "https://jsonplaceholder.typicode.com/users/1", true);

      xhr.onreadystatechange = function () {
        if (xhr.readyState === 4) { // request finished
          if (xhr.status === 200) { // success
            var data = JSON.parse(xhr.responseText);
            document.getElementById("result").innerHTML =
              "Name: " + data.name + "<br>" +
              "Email: " + data.email + "<br>" +
              "City: " + data.address.city;
          } else {
            document.getElementById("result").innerHTML =
              "Error: " + xhr.status;
          }
        }
      }
    }
  </script>
</body>
</html>
```

```
};

    xhr.send();    // no body needed for GET
}
</script>
</body>
</html>
```

Program: POST Request with XHR

```
var xhr = new XMLHttpRequest();
xhr.open("POST", "https://example.com/api/save", true);
xhr.setRequestHeader("Content-Type", "application/json");

xhr.onreadystatechange = function () {
    if (xhr.readyState === 4 && xhr.status === 201) {
        console.log("Saved:", xhr.responseText);
    }
};

var payload = JSON.stringify({ name: "Rahul", course: "MCA" });
xhr.send(payload);
```

The GET example fetches user data from a server without reloading the page and injects the parsed JSON into the DOM; the POST example shows sending JSON data to a server endpoint, which is the pattern used for form submissions and data creation in AJAX-driven applications.

Who is full stack developer? Explain the layers of web development process in detail.

Answer:

A **full stack developer** is a software engineer who can work on both the client side (front end) and the server side (back end) of an application, along with the database that supports it. They understand how each layer of a web application works and how the layers interact, which lets them build, debug, and deploy a complete application independently rather than depending entirely on separate specialist teams.

Typical Skill Set of a Full Stack Developer

- **Front end:** HTML, CSS, JavaScript, and a modern framework (React/Angular/Vue).
- **Back end:** a server-side language/framework (Node.js, Django, Spring Boot, PHP, etc.) to implement business logic and APIs.
- **Database:** SQL (MySQL/PostgreSQL) and/or NoSQL (MongoDB) for data storage.
- **Version control:** Git for tracking and collaborating on code.
- **Basic DevOps:** deployment, containers (Docker), and cloud hosting basics.
- **Soft skills:** problem solving and understanding the product end-to-end, since they interact with UI/UX, business logic and data all at once.

Layers of the Web Development Process

A typical full stack web application follows a layered (n-tier) architecture, where each layer has a distinct responsibility and communicates only with the layer directly above/below it:

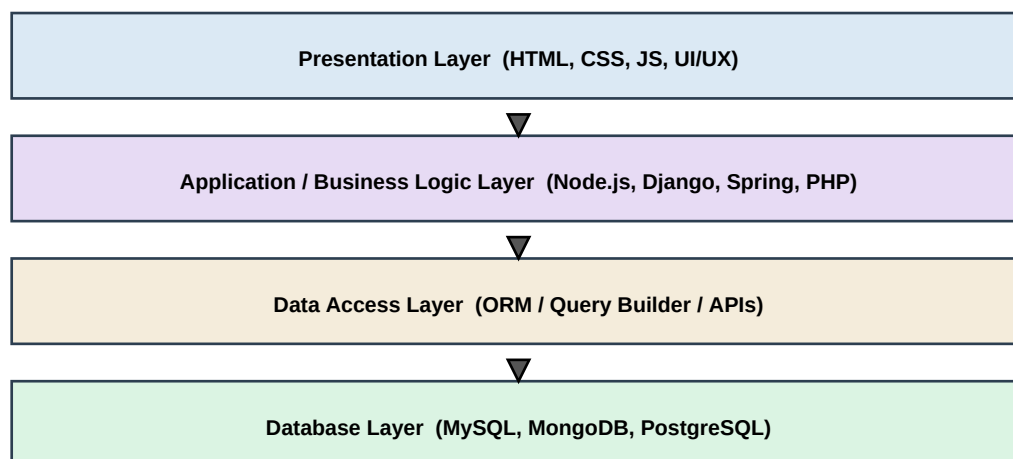


Fig. 2 — Layers of the web development process (n-tier architecture)

Layer	Responsibility
Presentation Layer (Front End)	What the user sees and interacts with in the browser — HTML for structure, CSS for styling, JavaScript/React/Angular for interactivity and UX.
Application / Business Logic Layer	Processes requests, applies business rules, performs validation and authentication, and coordinates data flow. Built with frameworks like Express.js, Django, or Spring Boot.
Data Access Layer	Sits between business logic and the database — handles queries, ORM mapping (e.g. Sequelize, Hibernate, Mongoose), and abstracts raw database operations.
Database Layer	Persistent storage for application data — relational (MySQL, PostgreSQL) or NoSQL (MongoDB, Redis) databases.

This layered separation makes applications easier to maintain, test and scale: each layer can be updated or replaced (e.g. swapping MySQL for MongoDB) with minimal impact on the others, as long as the interface between layers stays consistent.

What is the difference between SVG and canvas? Write a program to create a circle using canvas and fill the combination of three colors (red, blue and green) inside circle.

Answer:

Both **SVG** and **Canvas** are HTML5 technologies used to draw graphics on a web page, but they work in fundamentally different ways: SVG is a **vector-based, declarative (XML markup)** technology where each shape is an individual DOM element, while Canvas is a **raster-based, procedural (JavaScript)** technology where pixels are drawn onto a single bitmap surface.

SVG vs Canvas — Detailed Comparison

Aspect	SVG	Canvas
Type	Vector graphics (XML-based markup)	Raster/bitmap graphics (pixel based)
Rendering	Declarative — you describe shapes as elements in the DOM	Procedural — you draw pixels using JavaScript API calls
DOM Presence	Each shape is a separate DOM element (e.g. <circle>, <rect>)	The whole drawing is one single <canvas> element; individual shapes are not tracked
Scalability	Scales to any size/resolution without losing quality (resolution-independent)	Becomes blurry/pixelated when scaled up, since it is pixel-based
Event Handling	Each element supports its own event listeners (click, hover, etc.)	No per-shape events — you must calculate coordinates manually to detect interaction
Performance	Slower with a very large number of elements (DOM overhead)	Faster for a large number of objects / frequent repaint (e.g. games, animations)
Editability	Shapes can be modified/removed individually via DOM/CSS/JS	Once drawn, individual shapes cannot be selected — the whole canvas may need to be redrawn
Best suited for	Icons, logos, charts, diagrams, maps — where scalability and interactivity per-shape matter	Games, real-time data visualisation, image editing, animations with many objects
Accessibility	Better — elements are part of the DOM and can be styled/read by screen readers	Weaker — content is just pixels, with no inherent semantic structure

Program: Drawing a Circle on Canvas with Red, Blue & Green Fill

The circle below is filled using a radial gradient that blends red, blue and green together inside it, demonstrating a combination of the three colors within a single canvas-drawn circle:

```
<!DOCTYPE html>
<html>
<head><title>Canvas Circle - RGB Fill</title></head>
<body>
  <h2>Circle Filled with Red, Blue and Green (Canvas)</h2>
  <canvas id="myCanvas" width="300" height="300"
    style="border:1px solid #333;"></canvas>

  <script>
    var canvas = document.getElementById("myCanvas");
    var ctx = canvas.getContext("2d");

    var centerX = 150, centerY = 150, radius = 100;

    // Create a radial gradient that blends the three colors
    var gradient = ctx.createRadialGradient(
      centerX, centerY, 5,          // inner circle (start)
```

```

        centerX, centerY, radius    // outer circle (end)
    );
    gradient.addColorStop(0,    "red");
    gradient.addColorStop(0.5, "green");
    gradient.addColorStop(1,    "blue");

    // Draw the circle path
    ctx.beginPath();
    ctx.arc(centerX, centerY, radius, 0, 2 * Math.PI);

    // Fill circle with the red-green-blue gradient
    ctx.fillStyle = gradient;
    ctx.fill();

    // Outline
    ctx.lineWidth = 2;
    ctx.strokeStyle = "#000";
    ctx.stroke();
</script>
</body>
</html>

```

Alternative: Three Overlapping Colored Circles (Additive Blend Look)

Another common way to show a 'combination' of red, blue and green is to draw three semi-transparent overlapping circles, so their overlap areas visibly mix:

```

var ctx = document.getElementById("myCanvas").getContext("2d");
ctx.globalCompositeOperation = "lighter";    // additive blending

ctx.fillStyle = "rgba(255,0,0,0.6)";        // red
ctx.beginPath(); ctx.arc(120,130,70,0,2*Math.PI); ctx.fill();

ctx.fillStyle = "rgba(0,255,0,0.6)";        // green
ctx.beginPath(); ctx.arc(180,130,70,0,2*Math.PI); ctx.fill();

ctx.fillStyle = "rgba(0,0,255,0.6)";        // blue
ctx.beginPath(); ctx.arc(150,190,70,0,2*Math.PI); ctx.fill();

```

With `globalCompositeOperation = "lighter"`, the overlapping regions add their colour values together, producing yellow, cyan, magenta and white-ish blends where the red, green and blue circles intersect.